

Université de Lille

Master 1 - Cloud Computing & Cybersécurité

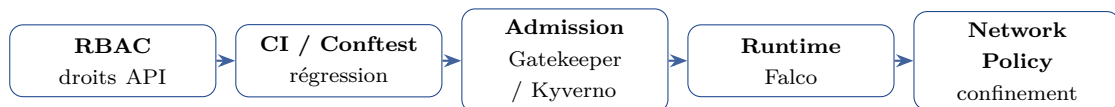
k8s-sec-lab

Laboratoire DevSecOps de sécurité Kubernetes

Projet Individuel (PJI) - Rapport technique final

Thèse principale

RBAC répond à une question d'autorisation : *qui peut appeler l'API Kubernetes ?*
Ce lab montre pourquoi cette réponse ne suffit pas à garantir que le workload envoyé au cluster est sûr.



Auteur	Youcef Boualili
Encadrant	Michaël Hauspie
Année	2025–2026
Dépôt	https://github.com/Boualili-Youcef/k8s-sec-lab

Résumé

Ce projet part d'une question concrète : un utilisateur Kubernetes peut-il rester dangereux même avec un RBAC limité ? Le lab démontre que oui. Un profil `dev-user` ne peut pas lister les secrets du namespace `dev-team`, mais il peut créer des pods. À partir de ce constat, trois scénarios contrôlés montrent qu'un manifest `privileged`, un montage `hostPath` et l'automount d'un token `ServiceAccount` restent déployables si aucune politique d'admission ne contrôle le contenu du workload.

Le projet met en place une réponse en couches successives et complémentaires. **Conftest** bloque les manifestes non conformes avant toute interaction avec le cluster. **Gatekeeper** et **Kyverno** refusent ces mêmes manifestes à l'admission Kubernetes. **Falco** détecte au runtime l'exécution d'un outil réseau et d'un shell après démarrage. Enfin, une **NetworkPolicy** ciblée bloque un flux sortant après admission.

Le scan de dix charts Helm montre que les mêmes règles peuvent aussi servir de filtre de revue avant l'adoption de composants open-source. Chaque affirmation est associée à un script de preuve rejouable, un log d'exécution, et un workflow GitHub Actions pour la régression CI.

Mots-clés : Kubernetes, RBAC, OPA, Conftest, Gatekeeper, Kyverno, Falco, NetworkPolicy, Policy-as-Code, DevSecOps, admission control, runtime security.

Table des matières

Résumé	i
1 Introduction et problématique	1
1.1 Contexte	1
1.2 Problème étudié	1
1.3 Objectifs	1
1.4 Structure du rapport	2
2 Architecture de la chaîne de contrôle	3
2.1 Vue d'ensemble	3
2.2 Description des couches	5
2.2.1 RBAC	5
2.2.2 Validation statique avec Conftest/OPA	5
2.2.3 Admission control avec Gatekeeper et Kyverno	5
2.2.4 Détection runtime avec Falco	5
2.2.5 Confinement réseau avec NetworkPolicy	5
2.3 Structure du dépôt	6
3 Méthodologie et environnement technique	7
3.1 Approche générale	7
3.2 Environnement technique	7
3.2.1 Cluster local Kind	7
3.2.2 Identité développeur	7
3.2.3 Versions des outils	8
3.3 Déroulement du projet	8
4 Résultats	9
4.1 Vérification du point de départ RBAC	9
4.2 Scénario A : conteneur privileged	9
4.3 Scénario B : montage hostPath	10
4.4 Scénario C : token ServiceAccount	10
4.5 Validation statique avec Conftest	11
4.6 Admission control avec Gatekeeper	12
4.7 Admission control avec Kyverno	12

4.8	CI GitHub Actions et livraison locale	13
4.9	Limite runtime et détection Falco	13
4.9.1	Preuve de la limite de l'admission	13
4.9.2	Détection Falco	14
4.10	Confinement réseau avec NetworkPolicy	14
4.11	Scan de charts Helm	15
5	Analyse et discussion	17
5.1	Analyse comparative des couches de sécurité	17
5.2	Ce que le projet prouve	17
5.3	Gatekeeper et Falco : deux contrôles complémentaires	18
5.4	Gatekeeper vs Kyverno	19
5.5	Positionnement du projet dans l'écosystème	19
6	Limites et perspectives	20
6.1	Limites du projet	20
6.2	Perspectives d'amélioration	20
7	Conclusion	22
A	Correspondance entre affirmations et preuves	23
B	Commandes clés de démonstration	24
C	Extrait de politique Rego	25

Table des figures

2.1	Fil conducteur du lab : du droit API au confinement réseau	3
2.2	Contrôles avant création effective du pod	4
2.3	Contrôles après admission : visibilité runtime et confinement réseau . .	4
4.1	Répartition des findings du scan Helm par type de contrôle	16

Liste des tableaux

2.1	Répertoires principaux du dépôt k8s-sec-lab	6
3.1	Versions des outils utilisés dans le lab	8
4.1	Résultats du scan Helm	15
5.1	Comparaison des couches de sécurité du projet	17
5.2	Comparaison Gatekeeper / Falco	18
A.1	Table de correspondance entre affirmations et preuves	23

Chapitre 1

Introduction et problématique

1.1 Contexte

Dans un cluster Kubernetes de production, le contrôle d'accès repose principalement sur le mécanisme RBAC (*Role-Based Access Control*). Cette couche répond à une question précise : *qui a le droit d'appeler l'API Kubernetes et sur quelles ressources ?* Elle est indispensable, mais elle ne juge pas le contenu sémantique d'un manifest soumis par un utilisateur autorisé.

Un développeur peut donc rester autorisé à créer un pod tout en soumettant une configuration risquée : un conteneur **privileged**, un montage direct du système de fichiers de l'hôte, ou un token **ServiceAccount** automatiquement injecté sans nécessité. Ces configurations sont des vecteurs d'escalade de privilèges bien documentés dans les référentiels tels que les benchmarks CIS Kubernetes et la documentation officielle de sécurité de Kubernetes.

1.2 Problème étudié

Ce projet s'intéresse à trois familles de risques volontairement simples mais pédagogiquement représentatives :

1. **Conteneur privileged** : accès étendu aux périphériques du nœud hôte.
2. **Volume hostPath** : visibilité directe sur le système de fichiers du nœud.
3. **AutomountServiceAccountToken** : identité Kubernetes injectée exploitable depuis le conteneur.

Ces trois cas suffisent à prouver qu'un contrôle d'autorisation ne remplace pas un contrôle de conformité du workload.

1.3 Objectifs

Les objectifs poursuivis sont les suivants :

- O1.** Démontrer expérimentalement la limite de RBAC seul sur un cluster local.
- O2.** Reproduire les trois scénarios d'attaque à partir d'un profil **dev-user** restreint.
- O3.** Bloquer ces scénarios avec des politiques Conftest, Gatekeeper et Kyverno.
- O4.** Prouver qu'un workload conforme reste autorisé après mise en place des contrôles.
- O5.** Montrer la limite de l'admission face à un comportement runtime, puis détecter ce comportement avec Falco.
- O6.** Bloquer un flux réseau sortant ciblé avec une NetworkPolicy après admission.
- O7.** Relier ces preuves à une chaîne CI/CD reproductible et à un scan statique de charts Helm.

1.4 Structure du rapport

Le rapport suit la progression logique du projet. Le chapitre 2 présente l'architecture de la chaîne de contrôle. Le chapitre 3 décrit la méthodologie et l'environnement technique. Le chapitre 4 expose les résultats par couche de sécurité. Le chapitre 5 propose une analyse comparative. Le chapitre 6 identifie les limites et les perspectives d'amélioration.

Chapitre 2

Architecture de la chaîne de contrôle

2.1 Vue d'ensemble

La logique du projet suit le parcours d'un manifest Kubernetes depuis sa création jusqu'à l'exécution du pod. Ce parcours comporte plusieurs points de contrôle successifs, chacun adressant une catégorie de risque différente.

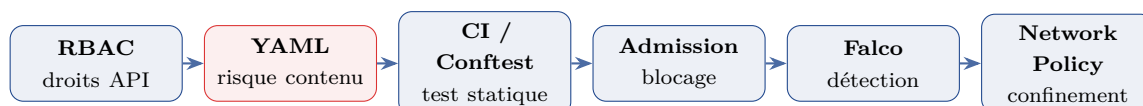


Figure 2.1 – Fil conducteur du lab : du droit API au confinement réseau

La première figure isole volontairement les contrôles qui se produisent avant la création du pod. La CI réduit le risque dans le dépôt, mais elle peut être contournée par un `kubectl apply` direct. C'est pour cette raison que RBAC et l'admission restent nécessaires dans le cluster.

La seconde figure sépare clairement deux rôles souvent confondus. Falco améliore la visibilité en détectant les comportements runtime, tandis qu'une NetworkPolicy agit sur le trafic réseau et peut bloquer un flux sortant ciblé.

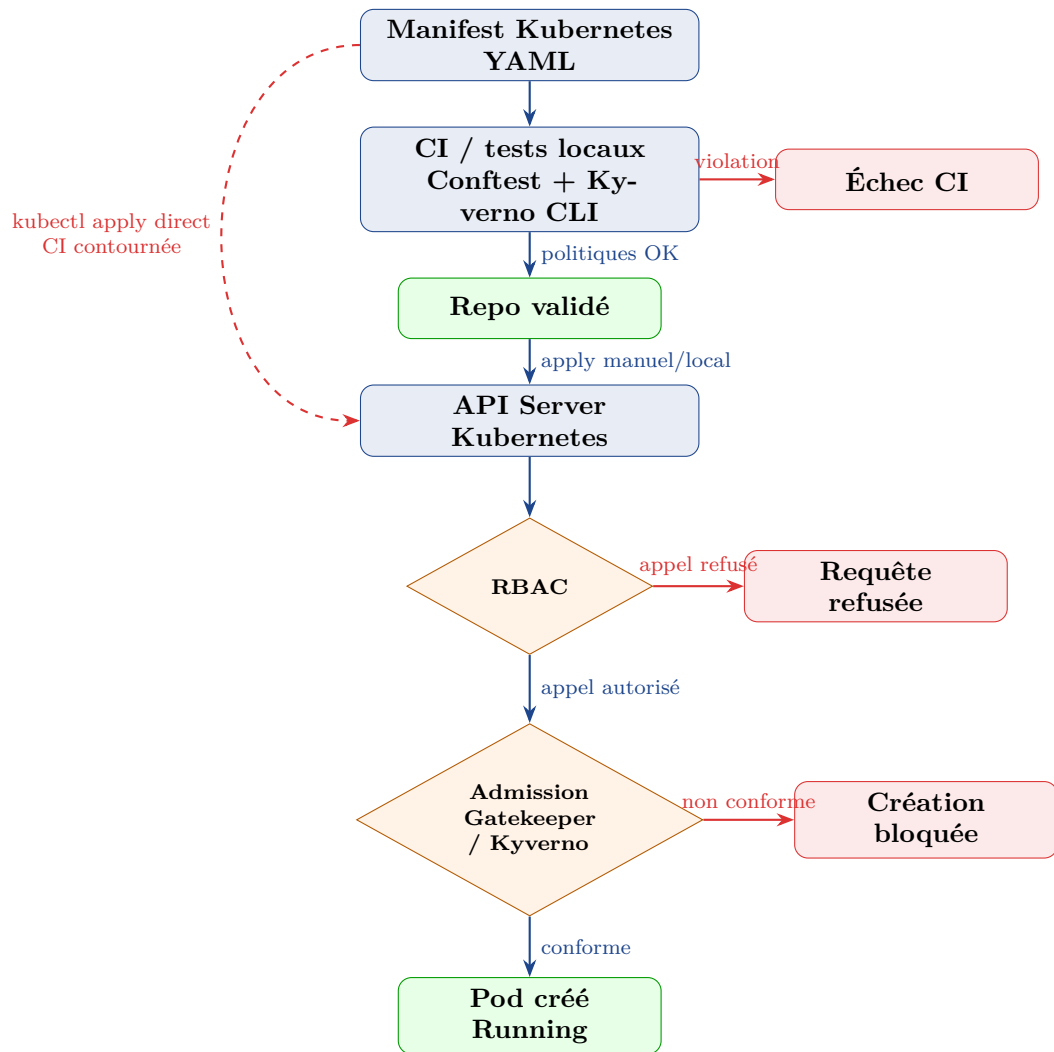


Figure 2.2 – Contrôles avant création effective du pod

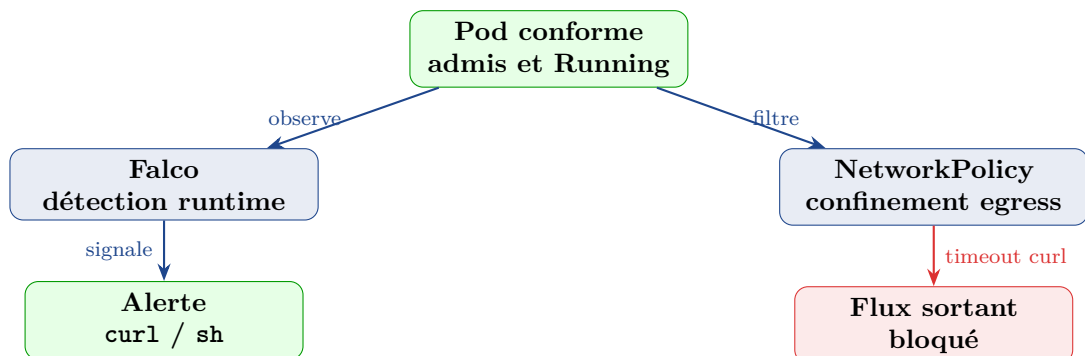


Figure 2.3 – Contrôles après admission : visibilité runtime et confinement réseau

2.2 Description des couches

2.2.1 RBAC

RBAC est la première couche d'autorisation. Elle répond à la question : *ce profil a-t-il le droit d'appeler cette API ?* Dans le projet, `dev-user` est autorisé à créer des pods dans `dev-team` mais pas à lire les secrets. Cette asymétrie constitue le point de départ de la démonstration.

2.2.2 Validation statique avec Conftest/OPA

Conftest évalue des règles Rego sur les fichiers YAML avant tout contact avec le cluster. Cette couche peut s'intégrer à une pipeline CI/CD et offre un retour immédiat au développeur. Elle ne remplace pas l'admission, car un développeur peut toujours contourner la CI en appliquant directement un manifest.

2.2.3 Admission control avec Gatekeeper et Kyverno

L'admission control opère à l'intérieur du cluster, après que RBAC a autorisé la requête. Gatekeeper utilise OPA comme moteur de politiques via des `ConstraintTemplate` et des `Constraint`. Kyverno utilise un modèle natif Kubernetes via des `ClusterPolicy`. Dans ce lab, les deux approches refusent les mêmes trois scénarios.

2.2.4 Détection runtime avec Falco

Falco surveille les appels système des conteneurs via un module noyau. Il détecte des comportements après démarrage qui ne sont pas visibles à l'admission : exécution d'un shell, lancement d'un outil réseau, accès à des fichiers sensibles. Il alerte mais ne bloque pas par lui-même.

2.2.5 Confinement réseau avec NetworkPolicy

Une `NetworkPolicy` Kubernetes définit des règles de pare-feu au niveau des pods. Dans le projet, une règle ciblée bloque tout le trafic sortant d'un pod de démonstration. Le pod est admis et démarre normalement, mais ses connexions réseau sortantes échouent.

2.3 Structure du dépôt

Table 2.1 – Répertoires principaux du dépôt k8s-sec-lab

Répertoire	Contenu
attack-manifests/	Manifests de démonstration des scénarios d’attaque
policies/conftest/	Règles Rego pour Conftest
policies/opa/	ConstraintTemplates et Constraints Gatekeeper
policies/kyverno/	ClusterPolicies Kyverno
policies/falco/	Règles Falco custom
policies/network/	NetworkPolicy de démonstration
runtime-manifests/	Pods de preuve runtime et réseau
scripts/evidence/	Scripts rejouables générant les logs de preuve
test-manifests/	Fixtures de régression CI (Deployment, CronJob)
scan/	Scan statique de charts Helm
report/evidence/	Logs et captures d’écran de preuve
.github/workflows/	Workflow GitHub Actions

Chapitre 3

Méthodologie et environnement technique

3.1 Approche générale

Le projet suit une démarche de **preuve progressive**. Chaque couche de sécurité est d'abord absente afin d'observer librement le risque, puis activée afin de démontrer son efficacité. Cette approche permet d'associer chaque contrôle à un résultat observable et mesurable.

Chaque étape est documentée par :

- un script shell rejouable dans `scripts/evidence/`,
- un log d'exécution dans `report/evidence/logs/`,
- un fichier de bilan journalier dans `report/evidence/`.

3.2 Environnement technique

3.2.1 Cluster local Kind

Le lab s'appuie sur un cluster Kind (*Kubernetes in Docker*) créé à partir d'un fichier de configuration `cluster/kind-config.yaml`. Kind permet de simuler un cluster multi-nœuds sur une machine locale sans infrastructure cloud.

3.2.2 Identité développeur

Le profil `dev-user` est représenté par un kubeconfig dédié (`dev-user-kubeconfig.yaml`) et des ressources RBAC dans `cluster/rbac/` :

- un `ServiceAccount` `dev-user`,
- un `Role` autorisant la gestion des `Pods`, `Pods/exec` et `Pods/log`,
- aucun droit sur les `secrets`,
- un `RoleBinding` liant ce rôle au `ServiceAccount`.

3.2.3 Versions des outils

Table 3.1 – Versions des outils utilisés dans le lab

Outil	Version
Docker	29.4.3
Kind	v0.24.0
kubectl client	v1.35.3
Helm	v3.15.0-rc.2
Conftest / OPA	0.68.2 / 1.15.2
Gatekeeper	v3.22.2
Kyverno	v1.18.0
Kyverno CLI (CI)	v1.15.2
Falco (chart)	8.0.5
Falco (runtime)	0.43.1

3.3 Dérroulement du projet

Le projet a été conduit en plusieurs phases successives :

- Phase 1. Initialisation** : bootstrap du cluster Kind, création du namespace `dev-team` et du profil `dev-user`.
- Phase 2. Preuve RBAC** : vérification des droits du profil développeur et observation du risque.
- Phase 3. Scénarios d’attaque** : déploiement des trois manifests dangereux sous RBAC seul.
- Phase 4. Contrôles statiques** : installation de Conftest, écriture des règles Rego, validation locale.
- Phase 5. Admission Gatekeeper** : installation, application des politiques, preuve de blocage.
- Phase 6. Admission Kyverno** : installation, application des politiques, preuve de blocage.
- Phase 7. CI GitHub Actions** : workflow d’audit automatisé incluant Conftest, Kyverno CLI et rendu Helm.
- Phase 8. Runtime Falco** : installation Helm, règles custom, preuves de détection runtime.
- Phase 9. Confinement réseau** : NetworkPolicy ciblée, preuve de blocage egress.
- Phase 10. Scan Helm** : audit de charts open-source avec la même politique Rego.

Chapitre 4

Résultats

4.1 Vérification du point de départ RBAC

La première étape consiste à vérifier que `dev-user` est effectivement un profil restreint, mais pas inoffensif.

```
$ kubectl auth can-i list secrets -n dev-team
no

$ kubectl auth can-i create pods -n dev-team
yes

$ kubectl get secrets -n dev-team
Error from server (Forbidden): secrets is forbidden: ...
```

Listing 4.1 – Vérification des droits RBAC de `dev-user`

Résultat R1 : asymétrie RBAC

Le profil est limité : il ne peut pas accéder aux secrets. Mais il conserve assez de droits pour déployer une charge de travail. Le problème démontré par le lab n'est pas un RBAC totalement ouvert, mais l'absence de contrôle sur le *contenu* du manifest.

Preuves

```
report/evidence/logs/02-day2-rbac-proof.log
report/evidence/day-02-rbac-deliverable.md
```

4.2 Scénario A : conteneur privileged

Avec RBAC seul, `dev-user` peut créer `01-privileged-pod.yaml`. Le pod démarre, et `kubectl exec` montre un accès étendu aux périphériques du nœud via `/dev`.

```
$ kubectl apply -f attack-manifests/01-privileged-pod.yaml \
  --kubeconfig dev-user-kubeconfig.yaml
```

```
pod/privileged-pod created

$ kubectl exec privileged-pod -- ls /dev | head -20
autofs  block  bsg    bus    char   cpu    ...
```

Listing 4.2 – Observation du pod privileged sous RBAC seul

Résultat R2 : scénario A

RBAC autorise la création du pod mais ne bloque pas le mode `privileged`. Un conteneur `privileged` peut accéder aux périphériques de l'hôte, charger des modules noyau, et contourner les contrôles de sécurité du nœud.

Preuves

`report/evidence/logs/03-scenario-a-privileged.log`

4.3 Scénario B : montage hostPath

`dev-user` peut créer `attack-manifests/02-hostpath-mount.yaml`. Le pod lit ensuite `/host/etc/os-release`, ce qui prouve une visibilité directe sur le système de fichiers du nœud.

```
$ kubectl exec hostpath-pod -- cat /host/etc/os-release
NAME="Ubuntu"
VERSION="22.04.4 LTS (Jammy Jellyfish)"
...
```

Listing 4.3 – Lecture du système de fichiers hôte depuis un pod `hostPath`

Résultat R3 : scénario B

Le pod ne reste pas dans son périmètre d'exécution propre. Un volume `hostPath` crée un lien direct avec le système de fichiers de l'hôte, permettant la lecture de fichiers sensibles ou la modification de configurations système.

Preuves

`report/evidence/logs/04-scenario-b-hostpath.log`

4.4 Scénario C : token ServiceAccount

`dev-user` peut créer `03-sa-token-theft.yaml`. Le pod démarre avec un token `ServiceAccount` monté automatiquement, puis ce token est utilisé pour interroger l'API Kubernetes depuis l'intérieur du conteneur.


```
$ kubectl exec sa-token-pod -- \
    curl -sk -H "Authorization: Bearer $(cat /var/run/secrets/
        kubernetes.io/serviceaccount/token)" \
    https://kubernetes.default.svc/api/v1/namespaces/dev-team/
    pods
{
  "kind": "PodList",
  "apiVersion": "v1",
  ...
}
```

Listing 4.4 – Utilisation du token ServiceAccount depuis le conteneur

Résultat R4 : scénario C

Même sans `privileged` ni `hostPath`, un pod apparemment banal peut emporter une identité Kubernetes exploitable. Le contrôle attendu est l'imposition de `automountServiceAccountToken: false` quand ce token n'est pas nécessaire.

Preuves

report/evidence/logs/05-scenario-c-sa-token.log

4.5 Validation statique avec Conftest

Conftest refuse les trois manifests dangereux avant toute interaction avec le cluster, puis accepte `04-safe-pod.yaml`. Le dossier `test-manifests/` étend la démonstration à des `Deployment` et `CronJob`.

```
$ conftest test attack-manifests/ --policy policies/conftest/
FAIL - attack-manifests/01-privileged-pod.yaml - ...
    Privileged containers are not allowed
FAIL - attack-manifests/02-hostpath-mount.yaml - ...
    hostPath volumes are not allowed
FAIL - attack-manifests/03-sa-token-theft.yaml - ...
    automountServiceAccountToken must be explicitly set to false
PASS - attack-manifests/04-safe-pod.yaml
```

Listing 4.5 – Sortie Conftest sur les manifests d'attaque

Résultat R5 : Conftest

La CI sert de filet de sécurité avant le cluster. Elle ne remplace pas l'admission, mais elle repère les manifests non conformes plus tôt et limite les erreurs qui arrivent jusqu'à Kubernetes.

Preuves

```
report/evidence/logs/11-conftest.log  
report/evidence/logs/12-policy-regression-conftest.log
```

4.6 Admission control avec Gatekeeper

Gatekeeper est installé via Helm, les `ConstraintTemplate` et `Constraint` sont appliqués, puis les trois manifests dangereux sont rejetés à l'admission.

```
$ kubectl apply -f attack-manifests/01-privileged-pod.yaml \  
  --kubeconfig dev-user-kubeconfig.yaml  
Error from server (Forbidden): admission webhook "validation.  
  gatekeeper.sh" denied the request:  
  [disallow-privileged] Privileged containers are not allowed.
```

Listing 4.6 – Blocage Gatekeeper à l'admission

Le pod sain `04-safe-pod.yaml` reste autorisé et passe en état `Running`.

Résultat R6 : Gatekeeper

Gatekeeper ajoute le contrôle qui manque à RBAC : l'utilisateur garde le droit de créer un pod, mais Kubernetes refuse les configurations dangereuses avant la création effective.

Preuves

```
report/evidence/logs/07-gatekeeper-blocking-proof.log
```

4.7 Admission control avec Kyverno

Kyverno produit le même résultat fonctionnel que Gatekeeper. Les `ClusterPolicy` sont appliquées, les trois manifests dangereux sont refusés.

```
$ kubectl apply -f attack-manifests/02-hostpath-mount.yaml \  
  --kubeconfig dev-user-kubeconfig.yaml  
Error from server: admission webhook "validate.kyverno.svc-fail  
  " denied the request:
```

```
resource Pod/dev-team/hostpath-pod was blocked due to the
following policies
disallow-hostpath: hostPath volumes are not allowed
```

Listing 4.7 – Blocage Kyverno à l’admission

Résultat R7 : Kyverno

Le projet montre que les mêmes exigences de sécurité peuvent être appliquées de façon reproductible avec deux moteurs différents. Ce point est important pour une équipe qui doit choisir entre Gatekeeper et Kyverno selon ses contraintes opérationnelles.

Preuves

report/evidence/logs/09-kyverno-blocking-proof.log

4.8 CI GitHub Actions et livraison locale

Le workflow GitHub Actions `.github/workflows/security-audit.yml` exécute automatiquement :

- Conftest sur les manifests d’attaque et les fixtures de régression,
- Kyverno CLI sur les mêmes manifests,
- Rendu Helm de Falco pour valider la syntaxe du ruleset custom.

Limite identifiée

La livraison des politiques vers le cluster reste volontairement locale. Un runner GitHub distant ne peut pas joindre un cluster Kind tournant sur une machine de travail. Cette séparation CI/CD est un choix d’architecture sain et non une limitation cachée.

Preuves

`.github/workflows/security-audit.yml`
`scripts/evidence/12-policy-regression.sh`

4.9 Limite runtime et détection Falco

4.9.1 Preuve de la limite de l’admission

Le pod `runtime-blind-spot` est créé avec un manifest conforme à toutes les politiques d’admission. Il exécute ensuite `curl` pour appeler un service externe après démarrage.

```
# Le manifest est conforme -> admis sans erreur
$ kubectl apply -f runtime-manifests/05-runtime-blind-spot.yaml
pod/runtime-blind-spot created

# Mais le pod effectue un appel reseau apres demarrage
# -> aucune couche d'admission ne l'a vu
```

Listing 4.8 – Le pod runtime-blind-spot contourne l’admission

4.9.2 Détection Falco

Falco est installé via Helm dans le namespace `falco` avec un ruleset custom ciblé sur `dev-team`.

```
# Log Falco - outil reseau detecte
[18:42:07.000] NOTICE K8S Sec Lab Network Tool In Dev Namespace
  proc=curl pid=12847 container=runtime-blind-spot
  namespace=dev-team image=curlimages/curl:latest

# Log Falco - shell detecte
[18:43:15.000] NOTICE K8S Sec Lab Shell In Dev Namespace
  proc=sh pid=13012 container=runtime-shell-probe
  namespace=dev-team
```

Listing 4.9 – Détection Falco d’un outil réseau et d’un shell

Résultat R8 : Falco

Falco détecte effectivement l’exécution de `curl` et de `sh` après admission dans `dev-team`. La couche runtime complète la visibilité sécurité du cluster, mais elle produit une alerte : elle ne bloque pas seule le pod ni son trafic.

Preuves

```
report/evidence/logs/10-runtime-blind-spot.log
report/evidence/logs/14-falco-runtime-detection.log
```

4.10 Confinement réseau avec NetworkPolicy

Une `NetworkPolicy deny-all-egress-runtime-probe` est appliquée sur le namespace `dev-team` avec un sélecteur ciblant le pod de démonstration.

```
$ kubectl apply -f runtime-manifests/07-networkpolicy-egress-
  probe.yaml
```

```
pod/runtime-networkpolicy-probe created # pod admis et Running

# Le curl echoue malgre le demarrage reussi du pod
curl: (28) Connection timed out after 10000 milliseconds
exit code: 28
```

Listing 4.10 – NetworkPolicy bloque l’egress du pod de démonstration

Résultat R9 : NetworkPolicy

Une NetworkPolicy ciblée peut bloquer un flux sortant après admission sans casser le déploiement du pod. La différence entre détection (Falco) et blocage (NetworkPolicy) est ainsi démontrée concrètement.

Preuves

report/evidence/logs/15-networkpolicy-egress-proof.log

4.11 Scan de charts Helm

Le script `scan/scan-helm-charts.sh` applique la même politique Rego sur le rendu `helm template` de plusieurs charts open-source.

Table 4.1 – Résultats du scan Helm

Chart	privileged	hostPath	SA token
bitnami/redis	0	0	0
bitnami/postgresql	0	0	0
bitnami/wordpress	0	0	0
argo/argo-cd	0	0	8
prometheus-community/prometheus	0	3	4
bitnami/nginx	0	0	0
bitnami/kafka	0	0	0
bitnami/mongodb	0	0	0
bitnami/elasticsearch	4	0	0
bitnami/jenkins	0	0	1

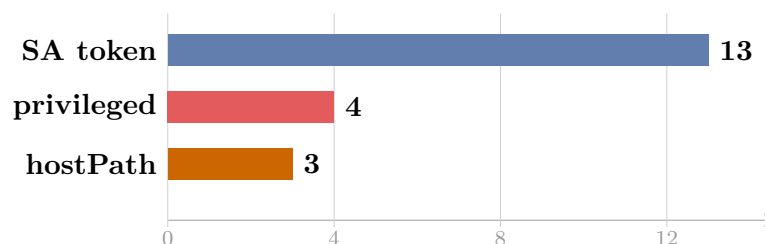


Figure 4.1 – Répartition des findings du scan Helm par type de contrôle

Le scan porte sur dix charts rendus avec leurs valeurs par défaut. Six charts ne déclenchent aucun finding sur les trois règles testées. Les quatre charts restants produisent vingt écarts à la politique : treize liés au token ServiceAccount, quatre à des conteneurs `privileged`, et trois à des volumes `hostPath`.

Cette lecture doit rester nuancée. Un finding ne signifie pas automatiquement que le chart est exploitable en production : il indique qu'un point de configuration doit être revu avant adoption. Les findings `SA token` apparaissent souvent lorsque le chart ne désactive pas explicitement l'automount. Les findings `hostPath` de Prometheus sont cohérents avec certains besoins d'observation bas niveau. Les findings `privileged` d'Elasticsearch peuvent correspondre à des conteneurs d'initialisation ou de configuration système. Le résultat utile pour une équipe plateforme est donc un tri rapide des composants qui nécessitent une justification, un durcissement ou une configuration dédiée.

Résultat R10 : scan Helm

Le scan Helm ajoute une dimension supply chain au projet. Une équipe plateforme peut réutiliser ses règles de sécurité pour pré-auditer des composants open-source avant adoption.

Preuves

```
scan/results/summary.md
scan/results/summary.json
scan/scan-helm-charts.sh
```

Chapitre 5

Analyse et discussion

5.1 Analyse comparative des couches de sécurité

Table 5.1 – Comparaison des couches de sécurité du projet

Couche	Moment	Ce qu'elle contrôle	Preuve dans le projet	Limite principale
RBAC	Avant création	Qui peut appeler l'API	dev-user peut créer des pods mais pas lire les secrets	Ne juge pas le contenu du workload
Conftest	CI / avant cluster	Conformité statique des manifests	3 échecs, 1 succès sur le pod sain	Pas d'état runtime, contournable
Gatekeeper	Admission	Conformité à l'API server	3 refus policy, pod sain admis	Comportement post-démarrage invisible
Kyverno	Admission	Conformité à l'API server	3 refus policy, pod sain admis	Même limite runtime
Falco	Après démarrage	Détection runtime via syscalls	Détecte curl et sh dans dev-team	Détecte mais ne bloque pas
NetworkPolicy	Après démarrage	Confinement réseau Kubernetes	Bloque l'egress du pod de démonstration	Ciblée, pas de micro-segmentation complète

5.2 Ce que le projet prouve

Le projet permet d'affirmer, avec preuves à l'appui :

- P1.** Un RBAC limité ne suffit pas à empêcher la création de workloads dangereux.
- P2.** Les trois risques choisis représentent une surface d’attaque Kubernetes réelle et documentée.
- P3.** Conftest permet de rejeter ces risques avant le cluster, dans la CI.
- P4.** Gatekeeper et Kyverno permettent de rejeter ces risques à l’admission.
- P5.** Un workload conforme reste autorisé : les politiques ne cassent pas l’usage légitime.
- P6.** L’admission seule ne couvre pas les comportements runtime après démarrage.
- P7.** Falco détecte l’exécution de `curl` et de `sh` après admission dans `dev-team`.
- P8.** Une NetworkPolicy ciblée peut bloquer un flux sortant après admission sans casser le déploiement.
- P9.** La logique Policy-as-Code peut être réutilisée pour l’audit de charts Helm et la CI.

5.3 Gatekeeper et Falco : deux contrôles complémentaires

Gatekeeper et Falco sont parfois rapprochés parce qu’ils participent tous les deux à la sécurité Kubernetes. Leur position dans la chaîne est pourtant différente : Gatekeeper bloque avant la création du pod, Falco observe ce qui se produit après le démarrage.

Table 5.2 – Comparaison Gatekeeper / Falco

Critère	Gatekeeper	Falco
Moment d’action	Admission Kubernetes, avant création effective du pod	Runtime, après démarrage du conteneur
Donnée observée	Objet Kubernetes soumis à l’API server : Pod, Deployment, CronJob	Événements système : processus, commandes, appels réseau, métadonnées conteneur
Action principale	Refuser la création d’un workload non conforme	Alerter lorsqu’un comportement suspect apparaît
Exemple du projet	Refus de <code>privileged</code> , <code>hostPath</code> et <code>token ServiceAccount</code> automatique	Détection de <code>curl</code> et de <code>sh</code> dans <code>dev-team</code>
Limite	Ne voit pas ce que le conteneur fera après démarrage	Détecte mais ne bloque pas seul la création ou le trafic réseau

Cette comparaison justifie l'ajout de la NetworkPolicy après Falco. Une alerte runtime améliore la visibilité, mais le confinement réseau fournit une mesure de réduction d'impact.

5.4 Gatekeeper vs Kyverno

Dans ce projet, Gatekeeper et Kyverno aboutissent au même résultat sur les trois scénarios. Le but n'est pas de choisir un vainqueur, mais de montrer deux façons d'appliquer la même exigence :

- **Gatekeeper** utilise OPA/Rego, langage généraliste, plus flexible mais avec une courbe d'apprentissage plus élevée.
- **Kyverno** utilise des ressources Kubernetes natives YAML/JSON, plus accessibles aux équipes platform engineering habituées aux manifests.

5.5 Positionnement du projet dans l'écosystème

Le projet couvre un sous-ensemble volontairement sélectionné des contrôles CIS Kubernetes Benchmark. Il ne vise pas l'exhaustivité mais la cohérence pédagogique : chaque contrôle est justifié par un scénario observable, chaque affirmation est associée à une preuve rejouable.

Chapitre 6

Limites et perspectives

6.1 Limites du projet

Limite identifiée

Les résultats sont à interpréter dans le périmètre suivant :

- Le cluster cible est un cluster Kind local, pas une plateforme de production multi-équipes.
- Trois familles de contrôles sont couvertes, pas le benchmark CIS complet.
- L'admission est comparée sur deux moteurs mais pas mesurée en performance ou coût opérationnel.
- Falco apporte une détection runtime, mais pas une réponse automatisée au blocage.
- La NetworkPolicy actuelle est ciblée sur un pod de démonstration, pas une micro-segmentation complète du namespace.
- Le scan Helm utilise des valeurs par défaut et une politique réduite à trois contrôles.
- La chaîne CD est volontairement locale : un runner GitHub ne peut pas joindre le cluster Kind local.

6.2 Perspectives d'amélioration

Les améliorations suivantes découlent directement des résultats du projet :

- A1.** Généraliser la stratégie NetworkPolicy : `deny-all` par défaut sur le namespace, puis flux DNS, API et applicatifs explicitement autorisés.
- A2.** Tester Cilium ou un autre CNI eBPF pour obtenir une visibilité réseau plus fine et des politiques L7.
- A3.** Ajouter des contrôles supply chain sur les images de conteneurs : Trivy ou Gype pour les vulnérabilités, Cosign pour la signature, SBOM pour la traçabilité.

- A4.** Étendre les règles Falco et prévoir une chaîne de réponse : écriture dans `/etc`, modification de `cgroups`, shell interactif, puis notification ou quarantaine via `Falcosidekick`.
- A5.** Ajouter un `Makefile` ou un `Taskfile` avec des cibles `bootstrap`, `test`, `demo`, `report` pour simplifier la reproductibilité.
- A6.** Automatiser la génération du rapport PDF et publier les logs de preuve comme artefacts GitHub Actions ou release GitHub.
- A7.** Intégrer un scan de posture RBAC avec `rbac-tool`, `rakness` ou un outil équivalent.
- A8.** Rejouer le lab sur un cluster managé afin de comparer les différences avec Kind : CNI réel, politiques réseau effectives, intégration IAM et logs centralisés.

Chapitre 7

Conclusion

Ce projet montre qu'en Kubernetes, le contrôle d'accès et le contrôle de conformité répondent à deux problèmes distincts. RBAC limite ce qu'un utilisateur peut faire vis-à-vis de l'API, mais ne permet pas de juger si un manifest crée un risque. Les trois scénarios d'attaque, `privileged`, `hostPath` et token ServiceAccount, prouvent que ce manque a des conséquences concrètes et observables.

La réponse apportée est cohérente et progressive. Conftest filtre les erreurs en amont dans la CI. Gatekeeper et Kyverno bloquent les workloads non conformes à l'admission. Le pod sain reste déployable, ce qui montre que les politiques ne cassent pas l'usage normal. Falco complète la démonstration en détectant au runtime un outil réseau et un shell dans `dev-team`. Une NetworkPolicy ciblée bloque effectivement l'egress d'un pod de preuve.

Thèse principale

Le point important du projet est la progression des preuves. RBAC explique les droits API, Conftest contrôle les manifests avant le cluster, Gatekeeper et Kyverno bloquent à l'admission, Falco observe le runtime et NetworkPolicy limite l'egress. Chaque résultat peut être rejoué avec un script et vérifié dans un log.

Annexe A

Correspondance entre affirmations et preuves

Table A.1 – Table de correspondance entre affirmations et preuves

Affirmation	Preuve principale
dev-user ne peut pas lire les secrets mais peut créer des pods	logs/02-day2-rbac-proof.log
RBAC seul laisse passer un pod privileged	logs/03-scenario-a-privileged.log
RBAC seul laisse passer un volume hostPath	logs/04-scenario-b-hostpath.log
RBAC seul laisse passer un token ServiceAccount exploitable	logs/05-scenario-c-sa-token.log
Conftest bloque 3 manifests et accepte le pod sain	logs/11-conftest.log
Gatekeeper bloque 3 manifests et accepte le pod sain	logs/07-gatekeeper-blocking-proof.log
Kyverno bloque 3 manifests et accepte le pod sain	logs/09-kyverno-blocking-proof.log
L'admission ne voit pas le comportement après démarrage	logs/10-runtime-blind-spot.log
Falco détecte un outil réseau et un shell après admission	logs/14-falco-runtime-detection.log
Une NetworkPolicy bloque l'egress sans casser le pod	logs/15-networkpolicy-egress-proof.log
La CI audite les manifests et le CD reste local	.github/workflows/security-audit.yml
Les mêmes règles servent à auditer des charts Helm	scan/results/summary.md

Annexe B

Commandes clés de démonstration

```
# 1. Bootstrap cluster local
bash scripts/evidence/01-bootstrap-lab.sh

# 2. Preuves RBAC
bash scripts/evidence/02-day2-rbac-proof.sh

# 3. Scenarios d'attaque sous RBAC seul
bash scripts/evidence/03-scenario-a-privileged.sh
bash scripts/evidence/04-scenario-b-hostpath.sh
bash scripts/evidence/05-scenario-c-sa-token.sh

# 4. Installation et preuve Gatekeeper
bash scripts/evidence/06-gatekeeper-install-and-apply.sh
bash scripts/evidence/07-gatekeeper-blocking-proof.sh

# 5. Installation et preuve Kyverno
bash scripts/evidence/08-kyverno-install-and-apply.sh
bash scripts/evidence/09-kyverno-blocking-proof.sh

# 6. Regression CI complete (Conftest + Kyverno CLI)
bash scripts/evidence/12-policy-regression.sh

# 7. Falco runtime detection
bash scripts/evidence/13-falco-install-and-apply.sh
bash scripts/evidence/14-falco-runtime-detection.sh

# 8. NetworkPolicy egress blocking
bash scripts/evidence/15-networkpolicy-egress-proof.sh
```

Listing B.1 – Bootstrap du lab et commandes clés de démonstration

Annexe C

Extrait de politique Rego

L'extrait ci-dessous reprend les trois contrôles utilisés dans la démonstration Conf-test : refuser un conteneur `privileged`, refuser un volume `hostPath` et demander explicitement `automountServiceAccountToken: false`.

```
package main

# Regle 1 - Privileged containers
deny[msg] {
    input.spec.containers[_].securityContext.privileged == true
    msg := "Privileged containers are not allowed"
}

# Regle 2 - hostPath volumes
deny[msg] {
    input.spec.volumes[_].hostPath
    msg := "hostPath volumes are not allowed"
}

# Regle 3 - automountServiceAccountToken
deny[msg] {
    not input.spec.automountServiceAccountToken == false
    msg := "automountServiceAccountToken must be explicitly set
        to false"
}
```

Listing C.1 – Extrait de la politique Rego Conftest (kubernetes.rego)